NBA Shot Make Probability Prediction

Kavin Gulati · October 11, 2026

Summary

A Histogram Based Gradient Boosting model predicts the probability that an NBA shot goes in, reaching a log loss of 0.632 and a ROC AUC of 0.662 on a held out season.

Goal	Predict a make probability (0 to 1) for every shot, not just make or miss
Data	**425,719** training shots and **213,977** test shots
Model	Histogram-Based Gradient Boosting (scikit-learn) with **15 features**
Main metric	Log loss, which rewards well-calibrated probabilities
Result	Original model kept: log loss **0.632177**, **ROC AUC 0.662427**, accuracy **63.35%**
Key lesson	Extra features and tuning did not beat the simpler model
Tools	Python, pandas, scikit learn, Joblib, Google Colab

Project Overview

The aim was to estimate how likely each NBA shot is to go in, using player tracking data.
A simple make or miss label hides a lot. A wide open layup and a heavily contested long two are both "shots," but they differ greatly in difficulty. A probability captures that difference, so the model's job was to output a number between 0 and 1 for every attempt.
The project had two goals:

1. Build a model that produces useful, well calibrated probabilities.
2. Understand which basketball factors drive those predictions.

Dataset and Exploration

The data covers NBA regular season shot attempts, split into a labeled training set and an unlabeled test set.

Dataset	Shot attempts	Outcome known?	Purpose
Training	425,719	Yes	Fit and validate models
Testing	213,977	No	Generate final make probabilities

About 45.81% of training shots were made. This is the baseline any model has to beat.
The features chosen describe how hard a shot is:

- Shot distance from the basket
- Defender proximity and whether the shot was contested
- Shooter movement, such as speed before the shot
- Shot type
- Shot clock time remaining

To prepare the data, relevant columns were selected, categorical features were converted to numbers, and the training and test sets were aligned to share the same feature columns.

Modeling Approach

The final model is a Histogram Based Gradient Boosting Classifier from scikit learn, compared against a simple baseline.

How it works**.** Gradient boosting builds many small decision trees in sequence. Each new tree focuses on correcting the mistakes of the trees before it.

Why does it fit this problem? Basketball factors interact. Being closely guarded matters a lot on a long jump shot but less on a layup at the rim. Gradient boosting captures these interactions without assuming straight line relationships.

Setup:

- 15 processed input features
- Evaluated with accuracy, ROC AUC and log loss
- Season based validation: trained on one season and tested on another, to check that the model generalizes to shots it has never seen

Feature Engineering and Tuning

New combined features and hyperparameter tuning were tested to see if extra complexity would help. It did not meaningfully improve results.

Engineered feature	Idea behind it
Distance squared	Shooting success may drop faster at longer range
Defender pressure	A single measure of how tightly the shooter is guarded
Distance × defender	Defense may matter more at some distances than others
Speed × distance	Moving shots may be harder from farther out
Late shot clock	Rushed shots with little time left may be worse

Hyperparameters tuned: learning rate, number of boosting iterations, and maximum leaf nodes. The tuned model was marginally more accurate, but the original model had slightly better ROC AUC and log loss. Since log loss was the project's evaluation metric, the original model was kept.

Evaluation and Results

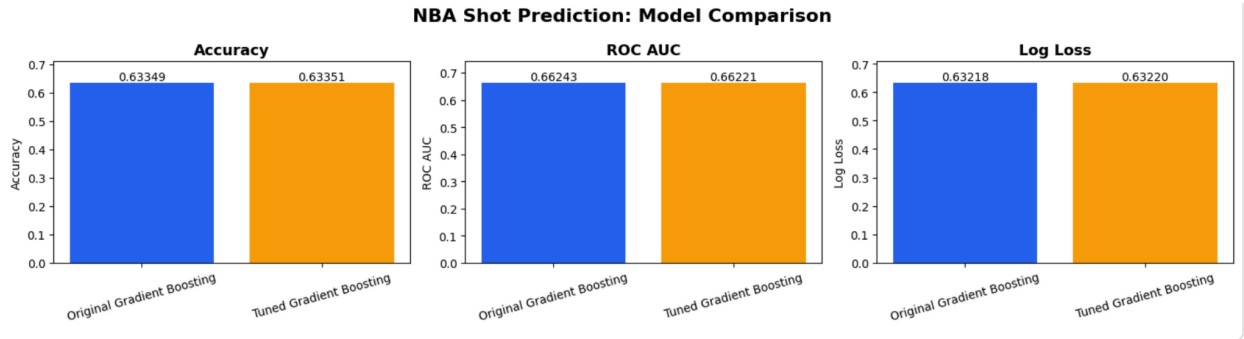
Both models performed almost identically. The original won on the two probability focused metrics, log loss and ROC AUC.
What each metric measures:

- Accuracy: share of shots correctly called made or missed.
- ROC AUC: how well the model ranks made shots above missed ones (0.5 is random, 1.0 is perfect).
- Log loss: how good the probabilities are. It heavily penalizes confident predictions that turn out wrong. Lower is better, and it was the main metric.

Metric	Original GB	Tuned GB	Better
Log loss (lower is better)	0.632177	0.632196	Original
ROC AUC (higher is better)	0.662427	0.662211	Original
Accuracy (higher is better)	63.3490%	63.3513%	Tuned

The gaps are tiny, in the fourth or fifth decimal place, so neither model is clearly better. The decision came down to the evaluation metric: log loss favored the original model.
For context, always guessing "miss" would be right about 54.2% of the time, so 63.3% accuracy is a real gain over the base rate.

Figure 1: Model performance comparison

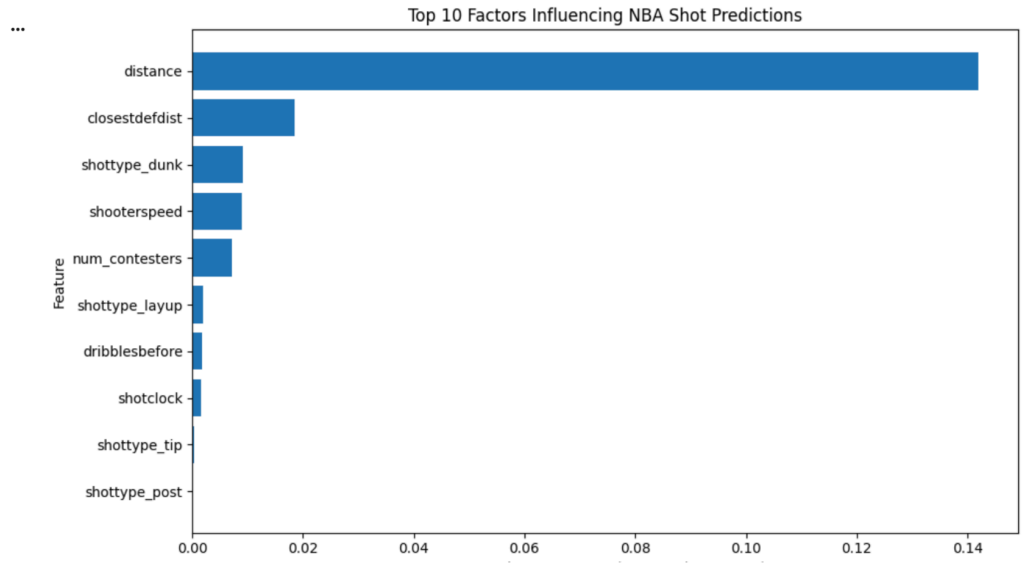


Feature Importance

Permutation importance was used to see which factors the model relies on most.
How it works: shuffle one feature's values, then measure how much the model's performance drops. A big drop means the model leaned heavily on that feature.

The features tested cover shot distance, defensive pressure, shot type, player movement and shot clock time. Each matches basketball intuition: shot difficulty depends on where the shot is taken, how closely it is guarded, and what happened just before it, not only on the shooter's skill.

Figure 2: Top 10 factors influencing shot predictions



Top 3 factors:

> ***Shot distance*****:** *This was by far the strongest factor, about 7 times more important than any other feature. Shots closer to the basket are much easier to make, so it makes sense that distance drives most of the prediction.*
>
> ***Closest defender distance*****:** *How much space the shooter has was the second biggest factor. A wide open shot is easier than one taken with a defender close by.*
>
> ***Dunk shot type*** **:** *Dunks are close to automatic, so knowing a shot was a dunk tells the model it will very likely go in.*

Final Predictions

The selected model produced a probability for all 213,977 test shots.

- **Training:** the original Gradient Boosting model was refitted on the available training data.
- **Submission file:** two columns, shot_id (shot identifier) and make_prob (probability between 0 and 1).
- **Saved model:** stored with Joblib. Reloading it was checked to give identical predictions, so results are reproducible.

Limitations and Next Steps

The biggest lesson was that more features and more tuning did not automatically improve the model.

Limitations:

- **Engineered features did not help.** Features built on sound basketball logic still did not raise performance, likely because gradient boosting already finds those interactions on its own.
- **Tuning gave tiny changes.** Differences between configurations were within the fourth decimal place.
- **Reused validation season.** The same validation season was used across many experiments, which can make results look slightly better than they would on new data.

Next steps:

1. Hold out a separate, untouched season for a final unbiased check.
2. Check calibration: when the model says 40%, do about 40% of those shots go in?
3. Test separate models by shot type or distance range.
4. With more data, add shooter tendencies, defender positioning and play context.

Conclusion

This project produced a working shot probability model and a clear lesson: simpler was just as good.

It covered the full machine learning workflow on a large sports dataset: preparing data, building and comparing models, engineering features, and evaluating results with several metrics.

The key takeaway is that accuracy alone does not tell the full story. When the goal is a probability, log loss is the better yardstick. Overall, the project shows how data can measure shot difficulty and support decision-making in basketball analytics.